Лабораторная работа № 4

Взаимодействие со стандартными приложениями Windows на VBScript/JScript

Дисциплина: «Операционные системы, среды и оболочки».

Студентка: _____. Группа: ИИ-551.

Преподаватель: _____. Дата: _____.

Цель работы

Научиться запускать приложения Windows из сценариев, передавать им клавиатурные команды, работать с файлами и выбирать параметры через диалоговые окна.

В работе используются Windows Script Host, языки VBScript/JScript, Калькулятор и Блокнот. WScript.Shell запускает приложения и управляет окнами, Scripting.FileSystemObject читает и записывает файлы.

Подготовка

Рабочая папка: C:\STUDENT\ИИ-551\Lab4. Для её создания и перехода в неё используются команды:

```
mkdir "C:\STUDENT\ИИ-551\Lab4"
cd /d "C:\STUDENT\ИИ-551\Lab4"
```

mkdir создаёт каталог. cd меняет текущую папку; ключ /d также меняет диск. Кавычки позволяют использовать пути с пробелами. Все файлы комплекта размещаются вместе.

Перед запуском закрываются лишние окна Калькулятора и Блокнота. Во время работы нельзя переключать окна: SendKeys отправляет команды активному приложению.

Задание 1. Вычисление 9 × 6

Сценарий script1.vbs запускает Калькулятор и Блокнот, вводит 9*6=, копирует ответ и вставляет его в Блокнот.

Option Explicit требует объявления переменных. Dim объявляет переменную, Set присваивает ссылку на объект. CreateObject("WScript.Shell") создаёт объект оболочки. Run запускает приложение: параметр 1 задаёт обычное окно, False позволяет продолжить сценарий без ожидания закрытия приложения.

AppActivate активирует окно по заголовку. Sleep задаёт паузу в миллисекундах, чтобы интерфейс успел отреагировать. SendKeys имитирует клавиатурный ввод. Записи ^c и ^v означают Ctrl+C и Ctrl+V, {ENTER} — Enter, %1 — Alt+1.

Процедуры CalcKeys и NoteKeys сначала активируют нужное окно, затем отправляют клавиши. Цикл For выполняет повторные попытки активации; Exit Sub выходит из процедуры после успеха. Если окно не найдено, MsgBox показывает сообщение, WScript.Quit 1 завершает сценарий с ошибкой.

```
Файл  Правка  Формат  Вид  Справка
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
If sh.Popup("Закройте другие окна Калькулятора и Блокнота. Переключите раскладку на английскую. Во время работы
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
sh.Run "notepad.exe", 1, False
WScript.Sleep 2000
CalcKeys "%1"
CalcKeys "{ESC}"
CalcKeys "9*6="
CalcKeys "^c"
NoteKeys "Результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"

Sub ActivateCalc()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Калькулятор") Or sh.AppActivate("Calculator") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Калькулятор. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
Sub ActivateNotepad()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Блокнот") Or sh.AppActivate("Notepad") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Блокнот. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
```

Рисунок 1 — Основная часть script1.vbs и ожидание активации окон.

```
    WScript.Quit 1
End Sub
Sub CalcKeys(keys)
    ActivateCalc
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
Sub NoteKeys(keys)
    ActivateNotepad
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
```

Рисунок 2 — Процедуры CalcKeys и NoteKeys.

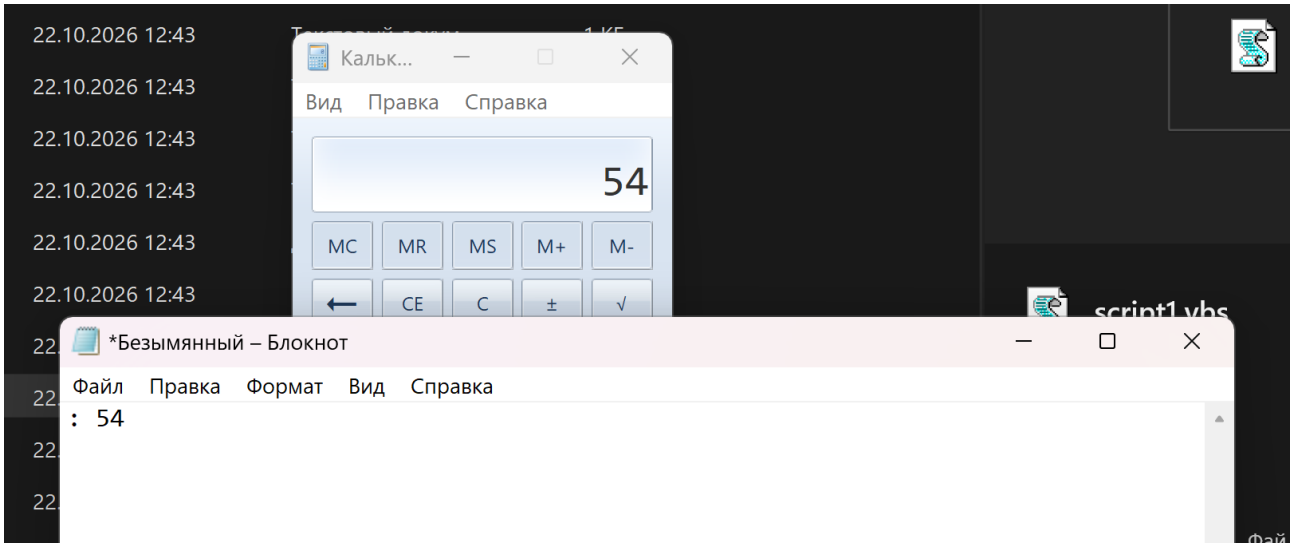


Рисунок 3 — Результат первого задания.

На дисплее Калькулятора и в Блокноте видно число 54. Проверка: 9 × 6 = 54. Русская подпись на снимке отображена неполно — это ошибка передачи текста, а не вычисления. В поздней версии подпись записывается в Unicode-файл, который открывается Блокнотом; повторный снимок этой версии здесь не представлен.

Задание 2. Сложение F7 и 7F

Сценарий script2.vbs выбирает режим «Программист», вводит F7+7F, переносит HEX-ответ в Блокнот, затем переключается на DEC и переносит десятичное представление. В конце возвращается обычный режим Калькулятора.

Команда %3 означает Alt+3 — режим «Программист», {F5} выбирает HEX, {F6} — DEC, %1 возвращает обычный режим. F7 без скобок — цифры шестнадцатеричного числа; {F7} — функциональная клавиша. Знак сложения записывается как {+}, потому что обычный + в SendKeys обозначает Shift. {ESC} очищает состояние Калькулятора.

```
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
If sh.Popup("Закройте другие окна Калькулятора и Блокнота. Переключите раскладку на английскую. Во время работ
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
sh.Run "notepad.exe", 1, False
WScript.Sleep 2000
CalcKeys "%3"
CalcKeys "{F5}"
CalcKeys "{ESC}"
CalcKeys "F7{+}7F="
CalcKeys "^c"
NoteKeys "Шестнадцатеричный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "{F6}"
CalcKeys "^c"
NoteKeys "Десятичный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "%1"

Sub ActivateCalc()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Калькулятор") Or sh.AppActivate("Calculator") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Калькулятор. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
```

Рисунок 4 — Основная часть script2.vbs.

```
End Sub
Sub ActivateNotepad()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Блокнот") Or sh.AppActivate("Notepad") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Блокнот. Закройте лишние окна и повторите
    WScript.Quit 1
End Sub
Sub CalcKeys(keys)
    ActivateCalc
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
Sub NoteKeys(keys)
    ActivateNotepad
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
```

Рисунок 5 — Процедуры активации окна и передачи клавиш.

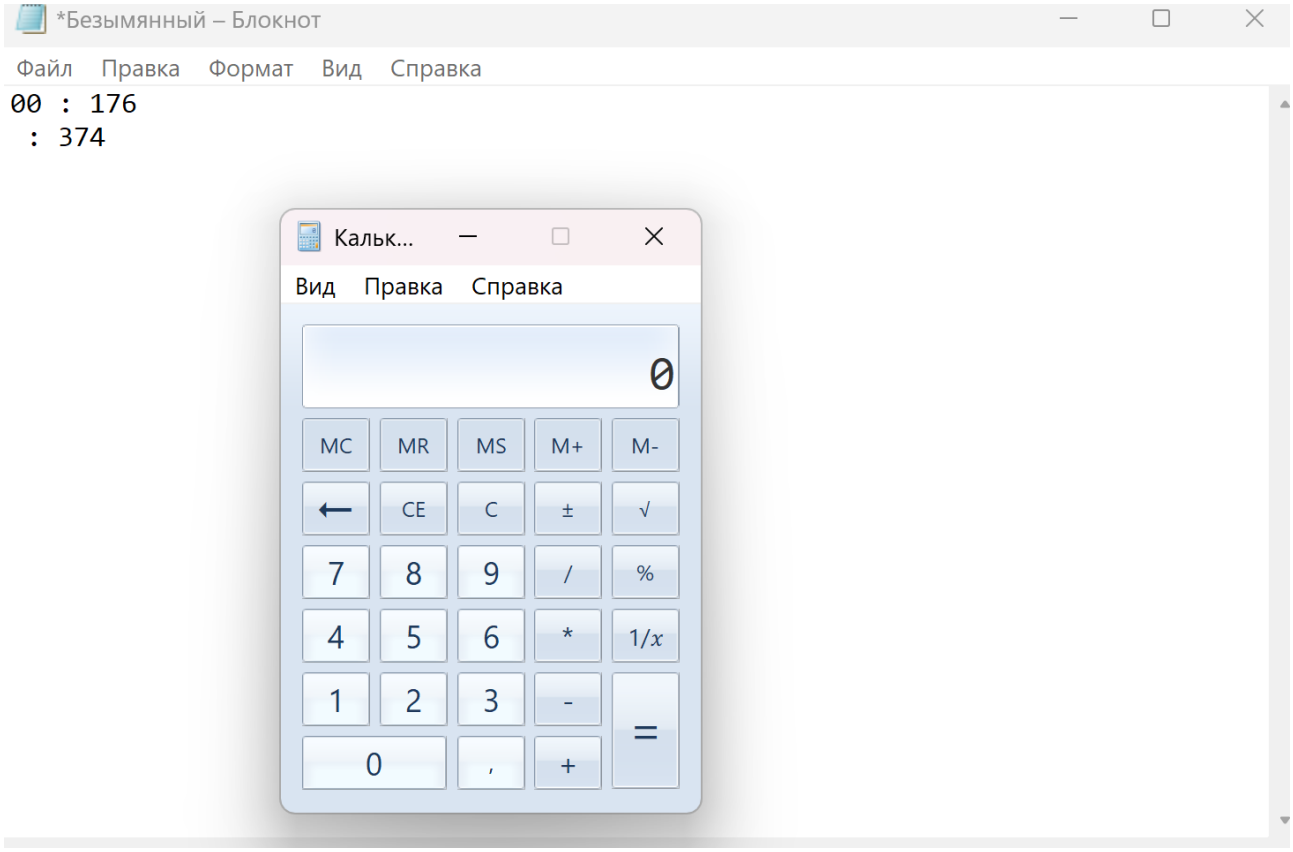


Рисунок 6 — Результат второго задания.

Проверка: $F7_{16} = 247_{10}$, $7F_{16} = 127_{10}$. Сумма равна $374_{10} = 176_{16}$. На снимке видны оба числа и обычный режим Калькулятора. Поясняющие подписи искажены; для полного соответствия заданию нужны строки «Шестнадцатеричный результат: 176» и «Десятичный результат: 374».

Задание 3. Вычисление выражений из файла

script3.js читает text1.txt построчно, разбирает выражения и записывает результаты в text2.txt. Поддерживаются четыре основания, дроби, операции +, −, *, / и круглые скобки.

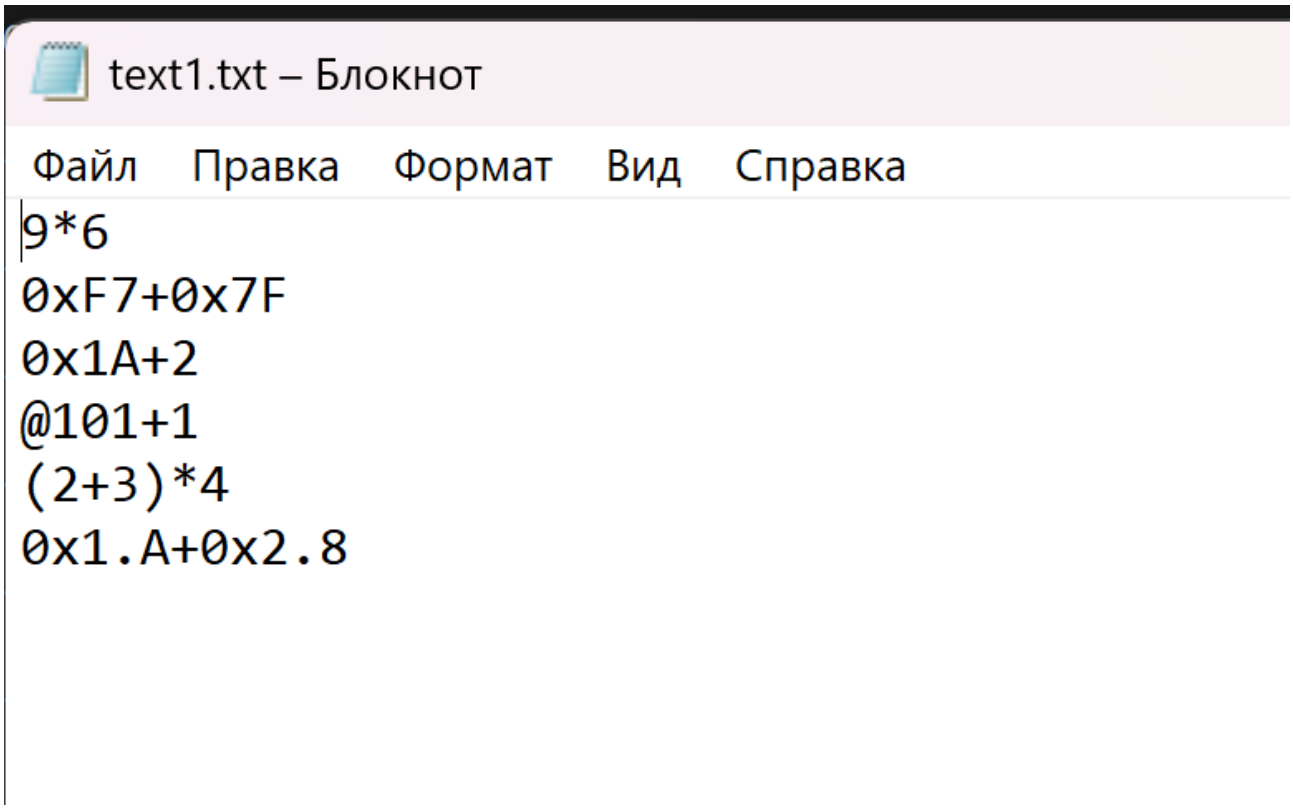


Рисунок 7 — Исходный файл text1.txt.

Запуск выполняется через run.cmd, пункт 3, или командой:

```
cscript.exe //nologo //E:JScript script3.js
```

cscript.exe запускает сценарий в консоли. //nologo убирает стартовый баннер, //E:JScript явно выбирает движок JScript. Это сценарий Windows Script Host, а не программа для браузера.

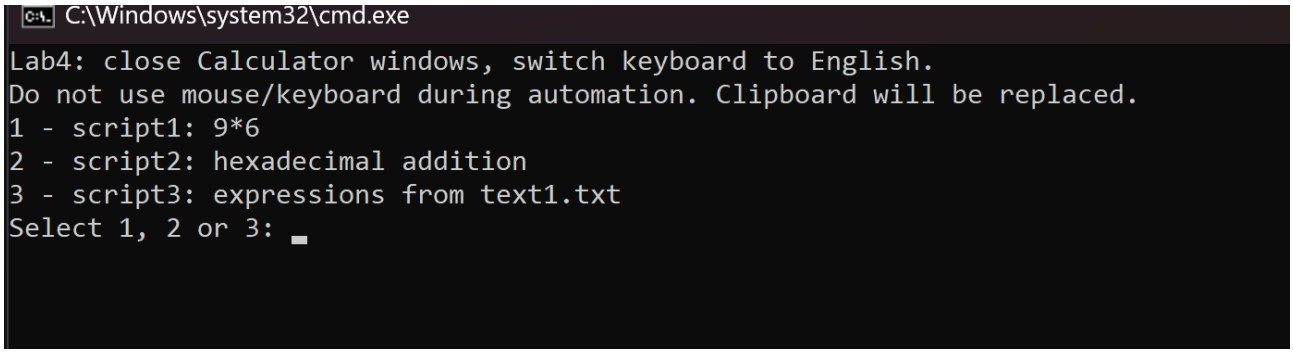


Рисунок 8 — Меню запуска сценариев.

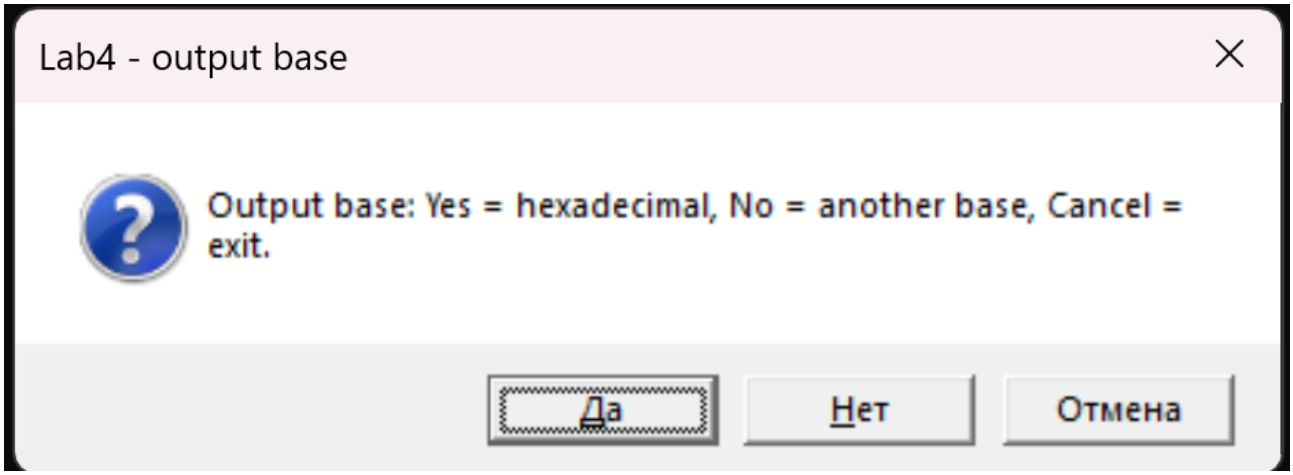


Рисунок 9 — Первый диалог выбора основания.

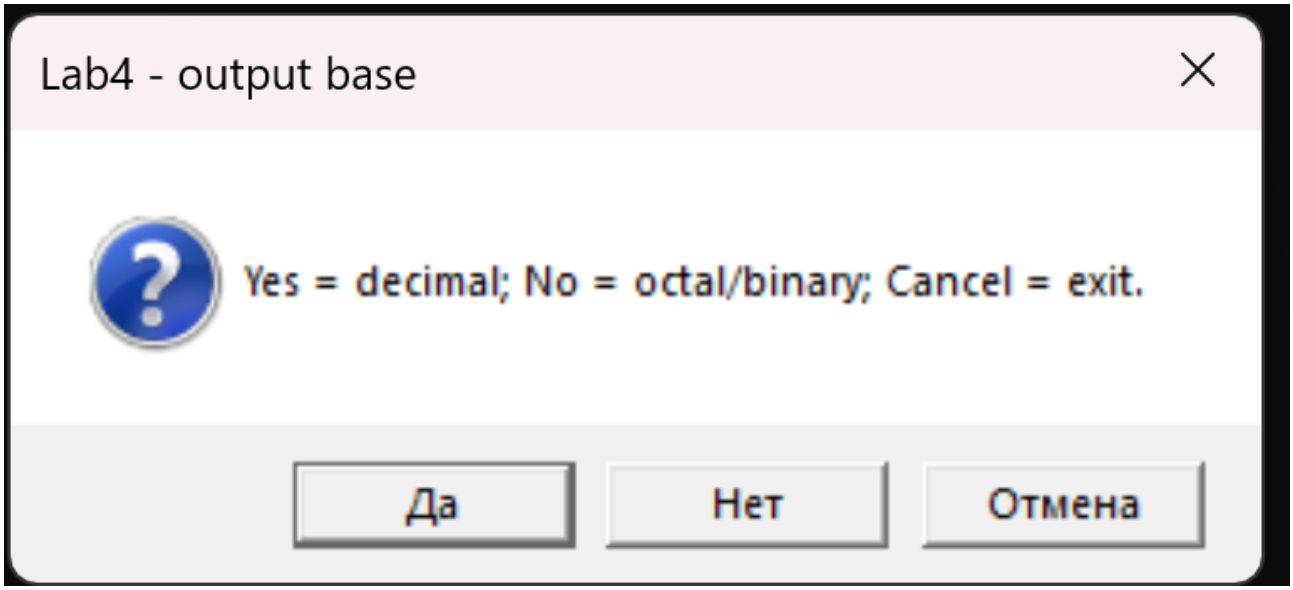


Рисунок 10 — Второй диалог выбора основания.

Popup показывает вопрос и возвращает код кнопки: 6 — «Да», 7 — «Нет», 2 — «Отмена». Для HEX выбирается «Да»; для DEC — «Нет → Да»; для OCT — «Нет → Нет → Да»; для BIN — «Нет → Нет → Нет». Параметр 3 + 32 задаёт кнопки «Да/Нет/Отмена» и значок вопроса.

Входные префиксы: 0x — шестнадцатеричное число, @ — двоичное, ведущий 0 перед другой цифрой — восьмеричное, без префикса — десятичное. Поэтому 010 равно восьми, а 0.5 считается десятичной дробью. Точка отделяет дробную часть.

GetParentFolderName(WScript.ScriptFullName) определяет папку сценария. FileExists проверяет наличие входа. OpenTextFile открывает его для чтения, ReadLine читает строку, AtEndOfStream показывает конец файла. CreateTextFile(output, true, true) создаёт или перезаписывает Unicode-файл. WriteLine записывает строку, Close закрывает поток.

Функция parse строит дерево выражения. number распознаёт число и проверяет цифры, atom обрабатывает скобки и унарные знаки, term — умножение и деление, expr — сложение и вычитание. Поэтому 2+3*4 даёт 14, а (2+3)*4 — 20.

Функция evaluate обходит дерево и вызывает bridge для бинарных операций. bridge создаёт временный файл запроса, запускает calculator.ps1 и читает ответ. PowerShell управляет Калькулятором и получает число с дисплея или через буфер обмена — в

зависимости от версии моста. JScript определяет порядок операций и переводит основания, а Калькулятор выполняет арифметику.

PowerShell запускается с -NoProfile без пользовательского профиля, -STA для используемого COM-окружения и -File с путём к скрипту. -Request и -Response — пути к файлам обмена. -ExecutionPolicy Bypass задаётся только для дочернего процесса и не меняет постоянную политику системы. shell.Run(..., 0, true) скрывает окно моста и ждёт завершения.

Функция format переводит число в выбранное основание. Целая часть преобразуется toString(base), дробная — последовательным умножением на основание. Вывод ограничен 12 дробными цифрами; [truncated] означает усечение остатка. Number использует double, поэтому произвольная точность не гарантируется.

Заголовок Output base относится к результатам справа от знака равенства. Префиксы слева относятся к исходным числам. Например, при основании 16 запись 9*6 = 36 верна: $54_{10} = 36_{16}$.

Ошибки обрабатываются через try/catch. Неверная строка получает отметку ERROR с номером, остальные продолжают обрабатываться. Код завершения 0 означает отсутствие ошибок, 1 — наличие хотя бы одной. Ошибка буфера обмена относится к получению ответа из приложения, а не к синтаксису примера.

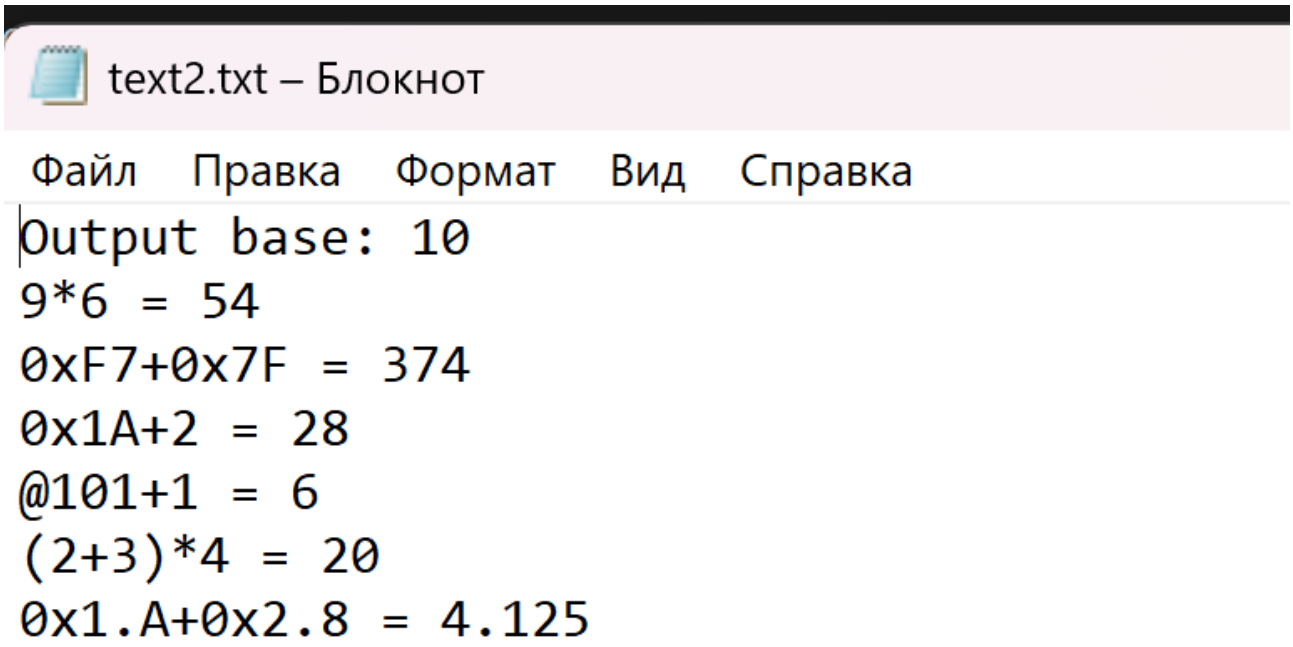


Рисунок 11 — Десятичный вывод шести выражений.

На снимке получены 54, 374, 28, 6, 20 и 4.125. Значения совпадают с проверочными: например, 0x1.A = 1.625, 0x2.8 = 2.5, сумма 4.125.

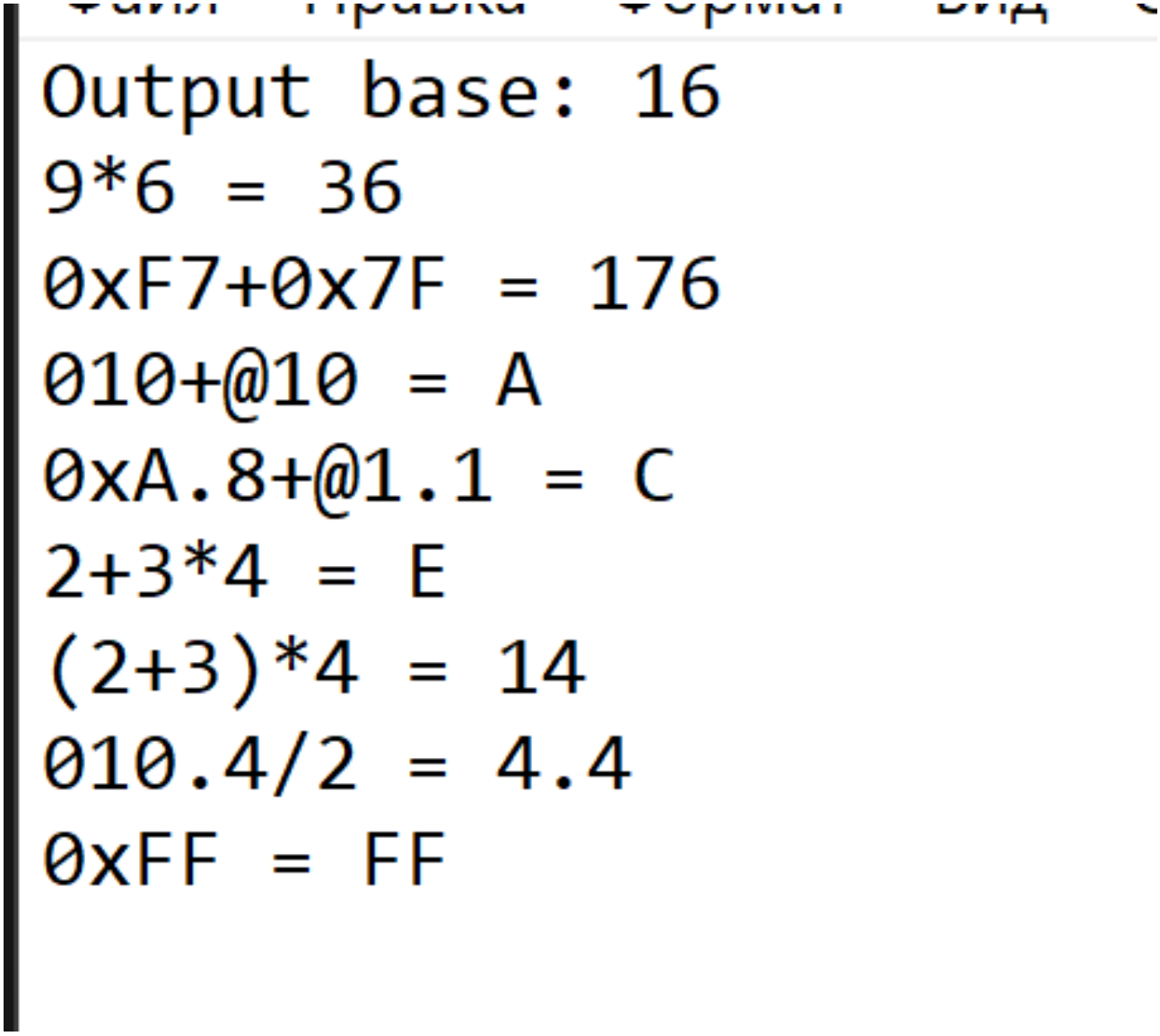


Рисунок 12 — Шестнадцатеричный вывод дополнительных примеров.

Показаны ответы 36, 176, A, C, E, 14, 4.4 и FF. Например, 010.4/2 = 8.5/2 = 4.25_{10} = 4.4_{16}. Снимков OCT- и BIN-вывода нет, поэтому их проверка здесь не заявляется.

Вывод

Изучены запуск приложений, активация окон, SendKeys, Popup и файловые операции. На снимках подтверждены числовые результаты первых двух заданий и показанные DEC-/HEX-результаты третьего. В первых двух заданиях выявлено искажение подписей. Для полного оформления требуются реальные снимки повторного вывода с правильным русским текстом.

Приложение. Код script3.js из исходного отчёта

Код сохранён для связи с пояснениями. Скриншоты выше относятся к фактическим запускам пользователя; подготовка этого документа не является новым запуском на Windows.

```
// Windows Script Host JScript (run with cscript.exe //nologo script3.js).
var fso = new ActiveXObject("Scripting.FileSystemObject");
var shell = new ActiveXObject("WScript.Shell");
var dir = fso.GetParentFolderName(WScript.ScriptFullName);
/* ES3: Windows Script Host JScript + Node.js test runner. No eval(). */
```

```
(function (root) {
    function fail(s) { throw new Error(s); }
    function parse(source) {
        var pos = 0, count = 0;
        if (source.length > 2000) fail("Expression exceeds 2000 characters");
        function ws() { while (/\s/.test(source.charAt(pos)) && pos < source.length) pos++; }
        function number() {
            ws(); var start = pos, base = 10, prefix = "";
            if (source.substr(pos, 2).toLowerCase() === "0x") { base = 16; pos += 2; prefix = "0x"; }
            else if (source.charAt(pos) === "@") { base = 2; pos++; prefix = "@"; }
            else if (source.charAt(pos) === "0" && /[0-9]/.test(source.charAt(pos + 1)) && pos + 1 <
    source.length) base = 8;
            var digits = "", dots = 0;
            while (pos < source.length && /[0-9a-fA-F.]/.test(source.charAt(pos))) {
                var c = source.charAt(pos++); if (c === ".") dots++;
                else if (parseInt(c, 16) >= base) fail("Invalid digit for base " + base + " at " + pos);
                digits += c;
            }
            if (!digits || digits === "." || dots > 1) fail("Invalid number at " + (start + 1));
            var parts = digits.split("."), n = 0, i, scale = base;
            for (i = 0; i < parts[0].length; i++) n = n * base + parseInt(parts[0].charAt(i), 16);
            if (parts.length > 1) for (i = 0; i < parts[1].length; i++) { n +=
    parseInt(parts[1].charAt(i), 16) / scale; scale *= base; }
            if (!isFinite(n) || Math.abs(n) > 9007199254740991) fail("Number exceeds safe numeric
    range");
            return {kind: "number", value: n};
        }
        function atom(depth) {
            if (depth > 64 || ++count > 500) fail("Expression too complex");
            ws(); var c = source.charAt(pos), a;
            if (c === "+" || c === "-") { pos++; return {kind: "unary", op: c, child: atom(depth + 1)};
    }
            if (c === "(") { pos++; a = expr(depth + 1); ws(); if (source.charAt(pos++) !== ")")
    fail("Missing closing parenthesis"); return a; }
            return number();
        }
        function term(depth) { var a = atom(depth), c; ws(); while ((c = source.charAt(pos)) === "*" ||
    c === "/") { pos++; a = {kind:"binary", op:c, left:a, right:atom(depth)}; ws(); } return a; }
        function expr(depth) { var a = term(depth), c; ws(); while ((c = source.charAt(pos)) === "+" ||
    c === "-") { pos++; a = {kind:"binary", op:c, left:a, right:term(depth)}; ws(); } return a; }
        var tree = expr(0); ws(); if (pos !== source.length) fail("Unexpected character at " + (pos +
    1)); return tree;
    }
    function evaluate(tree, calculator) {
        if (tree.kind === "number") return tree.value;
        if (tree.kind === "unary") return (tree.op === "-" ? -1 : 1) * evaluate(tree.child, calculator);
        var a = evaluate(tree.left, calculator), b = evaluate(tree.right, calculator);
        if (tree.op === "/" && b === 0) fail("Division by zero");
        var n = calculator(a, tree.op, b);
        if (!isFinite(n) || Math.abs(n) > 9007199254740991) fail("Result exceeds safe numeric range");
        return n;
    }
    function format(n, base) {
        if (base !== 2 && base !== 8 && base !== 10 && base !== 16) fail("Unsupported base");
        if (!isFinite(n) || Math.abs(n) > 9007199254740991) fail("Result exceeds safe numeric range");
        var sign = n < 0 ? "-" : "", x = Math.abs(n), whole = Math.floor(x), fraction = x - whole;
        var text = whole.toString(base).toUpperCase(), tail = "", i, d;
        for (i = 0; i < 12 && fraction > 0; i++) { fraction *= base; d = Math.floor(fraction); tail +=
    d.toString(base).toUpperCase(); fraction -= d; }
        tail = tail.replace(/0+$/, "");
        return sign + (base === 16 ? "0x" : base === 8 ? "0" : base === 2 ? "@" : "") + text + (tail ?
    "." + tail : "") + (fraction > 0 ? " [truncated]" : "");
    }
    var api = {parse:parse, evaluate:evaluate, format:format};
    if (typeof module !== "undefined" && module.exports) module.exports = api; else root.LabEngine =
    api;
}(this));

function quote(s) { return '"' + s + '"'; }
function bridge(mode, a, op, b) {
    var req = fso.BuildPath(fso.GetSpecialFolder(2), fso.GetTempName());
    var res = fso.BuildPath(fso.GetSpecialFolder(2), fso.GetTempName());
    var file = fso.CreateTextFile(req, true, true);
    file.WriteLine(mode); file.WriteLine(String(a)); file.WriteLine(op); file.WriteLine(String(b));
    file.Close();
    var code = shell.Run('powershell.exe -NoLogo -NoProfile -STA -ExecutionPolicy Bypass -File ' +
    quote(dir + '\\calculator.ps1') + ' -Request ' + quote(req) + ' -Response ' + quote(res), 0, true);
    fso.DeleteFile(req);
```

```
    if (!fso.FileExists(res)) throw new Error('PowerShell bridge failed to start');
    file = fso.OpenTextFile(res, 1, false, -1); var text = file.ReadAll(); file.Close();
    fso.DeleteFile(res);
    var lines = text.split('\r\n');
    if (code !== 0 || lines[0] !== 'OK') throw new Error(text);
    var value = Number(lines[1]); if (!isFinite(value)) throw new Error('Invalid Calculator response');
    return value;
}
var choice = shell.Popup('Output base: Yes = hexadecimal, No = another base, Cancel = exit.', 0, 'Lab4 -
     output base', 3 + 32);
if (choice === 2) WScript.Quit(0);
var base = 16;
if (choice === 7) {
    choice = shell.Popup('Yes = decimal; No = octal/binary; Cancel = exit.', 0, 'Lab4 - output base', 3
    + 32);
    if (choice === 2) WScript.Quit(0);
    if (choice === 6) base = 10;
    else {
        choice = shell.Popup('Yes = octal; No = binary; Cancel = exit.', 0, 'Lab4 - output base', 3 +
    32);
        if (choice === 2) WScript.Quit(0);
        base = choice === 6 ? 8 : 2;
    }
}
var input = dir + '\\text1.txt', output = dir + '\\text2.txt';
if (!fso.FileExists(input)) { WScript.Echo('Missing text1.txt'); WScript.Quit(1); }
// Input is ASCII (expressions only), output UTF-16LE for Russian Windows compatibility.
var reader = fso.OpenTextFile(input, 1, false, 0), results = [], lineNumber = 0, failures = 0;
while (!reader.AtEndOfStream) {
    var line = reader.ReadLine(); lineNumber++;
    if (/^\s*$/.test(line)) continue;
    try {
        var tree = LabEngine.parse(line), calls = 0;
        var result = LabEngine.evaluate(tree, function(a, op, b) { calls++; return bridge('binary', a,
    op, b); });
        if (!calls) result = bridge('identity', result, '', 0);
        results.push(line + ' = ' + LabEngine.format(result, base).replace(/^(-?)(0x|@)/,
    '$1').replace(/^(-?)0(?=[0-7])/, '$1'));
    } catch (e) { failures++; results.push('Line ' + lineNumber + ': ' + line + ' -> ERROR: ' +
    e.message); }
}
reader.Close();
var writer = fso.CreateTextFile(output, true, true);
writer.WriteLine('Output base: ' + base);
for (var i = 0; i < results.length; i++) writer.WriteLine(results[i]);
writer.Close();
WScript.Echo('Written: ' + output + '; expressions: ' + results.length + '; errors: ' + failures);
WScript.Quit(failures ? 1 : 0);
```